

온라인 쇼핑몰 시퀀스 다이어그램 설계서 v1

0. 문서 목적

이 문서는 온라인 쇼핑몰의 핵심 유스케이스를 시퀀스 다이어그램으로 정의한다.

대상 흐름은 다음과 같다.

1. 주문 생성 및 재고 예약
2. 결제 승인 성공
3. PG 콜백 중복 수신
4. 결제 실패 및 재고 예약 해제
5. 주문 취소 및 결제 취소
6. 리뷰 작성
7. 결제 대기 주문 만료 배치
8. 결제 상태 불일치 복구 배치

1. 주문 생성 및 재고 예약

1-1. 목적

주문 생성 시 실제 결제 완료 전까지는 주문을 `PENDING_PAYMENT` 상태로 생성하고, 재고는 예약 상태로 잡아둔다.

1-2. 핵심 규칙

- 주문 생성과 재고 예약은 하나의 DB 트랜잭션으로 처리한다.
- 여러 상품을 주문할 경우 `product_id` 오름차순으로 재고를 예약한다.
- 재고 부족 상품이 하나라도 있으면 전체 주문 생성을 실패시킨다.
- 외부 PG 호출은 DB 트랜잭션 안에서 수행하지 않는다.

1-3. 시퀀스 다이어그램

```
sequenceDiagram
    actor User
    participant Client
    participant OrderController
    participant OrderService
    participant ProductRepository
    participant InventoryRepository
    participant OrderRepository
    participant PaymentRepository
```

participant DB

User->>Client: 주문 요청

Client->>OrderController: POST /api/orders

OrderController->>OrderService: createOrder(memberId, request)

OrderService->>ProductRepository: 상품 목록 조회

ProductRepository->>DB: SELECT product WHERE product_id IN (...)

DB-->>ProductRepository: 상품 목록

ProductRepository-->>OrderService: 상품 목록 반환

OrderService->>OrderService: product_id 오름차순 정렬

loop 주문 상품별 재고 예약

OrderService->>InventoryRepository: reserve(productId, quantity)

InventoryRepository->>DB: UPDATE inventory SET reserved += q,
available -= q WHERE available >= q

DB-->>InventoryRepository: updated row count

InventoryRepository-->>OrderService: 예약 성공/실패

end

alt 재고 부족

OrderService-->>OrderController: OUT_OF_STOCK 예외

OrderController-->>Client: 400 OUT_OF_STOCK

Client-->>User: 재고 부족 안내

else 재고 예약 성공

OrderService->>OrderRepository: 주문 PENDING_PAYMENT 저장

OrderRepository->>DB: INSERT orders

OrderRepository->>DB: INSERT order_item

OrderRepository->>DB: INSERT delivery

OrderService->>PaymentRepository: 결제 READY 저장

PaymentRepository->>DB: INSERT payment

OrderService-->>OrderController: orderId, orderNo, paymentKey

OrderController-->>Client: 주문 생성 성공

Client-->>User: 결제 진행 화면 표시

end

2. 결제 승인 성공

2-1. 목적

사용자가 결제를 완료하면 서버는 PG 결제 승인 결과를 확인하고, 주문 상태를 **PAID**로 변경한다.

2-2. 핵심 규칙

- 결제 승인 API는 Idempotency-Key를 사용한다.
- 동일 결제 승인 요청이 중복되어도 한 번만 처리한다.
- 결제 금액과 주문 금액이 반드시 일치해야 한다.
- 결제 승인 성공 후 예약 재고를 확정 차감한다.
- 결제 완료 이벤트는 Outbox 테이블에 저장한다.

2-3. 시퀀스 다이어그램

```
sequenceDiagram
    actor User
    participant Client
    participant PaymentController
    participant PaymentService
    participant PG
    participant PaymentRepository
    participant OrderRepository
    participant InventoryRepository
    participant OutboxRepository
    participant DB

    User->>Client: 결제 완료
    Client->>PaymentController: POST /api/payments/approve + Idempotency-Key
    PaymentController->>PaymentService: approvePayment(request, idempotencyKey)

    PaymentService->>PaymentRepository: idempotencyKey로 기존 결제 조회
    PaymentRepository->>DB: SELECT payment WHERE idempotency_key = ?
    DB-->>PaymentRepository: 기존 결제 없음
    PaymentRepository-->>PaymentService: 없음

    PaymentService->>OrderRepository: 주문 조회
    OrderRepository->>DB: SELECT orders WHERE order_id = ?
    DB-->>OrderRepository: PENDING_PAYMENT 주문
    OrderRepository-->>PaymentService: 주문 반환

    PaymentService->>PaymentService: 주문 금액과 결제 요청 금액 검증

    PaymentService->>PG: 결제 승인 요청
    PG-->>PaymentService: 승인 성공

    PaymentService->>PaymentRepository: payment APPROVED 변경
    PaymentRepository->>DB: UPDATE payment SET status = APPROVED

    PaymentService->>OrderRepository: order PAID 변경
```

```

OrderRepository->>DB: UPDATE orders SET status = PAID

loop 주문상품별 재고 확정
    PaymentService->>InventoryRepository: confirmReservedStock(productId,
quantity)
    InventoryRepository->>DB: UPDATE inventory SET reserved -= q, total -
= q
end

PaymentService->>OutboxRepository: OrderPaidEvent 저장
OutboxRepository->>DB: INSERT outbox_event

PaymentService-->>PaymentController: 결제 승인 결과
PaymentController-->>Client: 200 OK
Client-->>User: 결제 완료 화면

```

3. PG 콜백 중복 수신

3-1. 목적

PG사는 결제 성공 콜백을 여러 번 보낼 수 있다.
서버는 중복 콜백을 안전하게 처리해야 한다.

3-2. 핵심 규칙

- payment_key 기준으로 결제 정보를 찾는다.
- 이미 APPROVED 상태이면 추가 처리하지 않는다.
- 중복 콜백에도 성공 응답을 반환한다.
- 콜백 처리 실패 시 PG사가 재시도할 수 있으므로 멍등해야 한다.

3-3. 시퀀스 다이어그램

```

sequenceDiagram
    participant PG
    participant PaymentCallbackController
    participant PaymentService
    participant PaymentRepository
    participant OrderRepository
    participant DB

    PG->>PaymentCallbackController: POST /api/payments/callback
    PaymentCallbackController->>PaymentService: handleCallback(callback)

```

```

PaymentService->>PaymentRepository: paymentKey로 결제 조회
PaymentRepository->>DB: SELECT payment WHERE payment_key = ?
DB-->>PaymentRepository: payment APPROVED
PaymentRepository-->>PaymentService: 결제 정보 반환

alt 이미 APPROVED 상태
    PaymentService->>PaymentService: 중복 콜백으로 판단
    PaymentService-->>PaymentCallbackController: 성공 처리
    PaymentCallbackController-->>PG: 200 OK
else 아직 처리되지 않은 콜백
    PaymentService->>PaymentRepository: payment APPROVED 변경
    PaymentRepository->>DB: UPDATE payment

    PaymentService->>OrderRepository: order PAID 변경
    OrderRepository->>DB: UPDATE orders

    PaymentService-->>PaymentCallbackController: 성공 처리
    PaymentCallbackController-->>PG: 200 OK
end

```

4. 결제 실패 및 재고 예약 해제

4-1. 목적

결제 실패 시 주문 상태를 `PAYMENT_FAILED` 로 변경하고 예약된 재고를 해제한다.

4-2. 핵심 규칙

- 결제 실패 사유를 `payment.failure_code`, `failure_message`에 저장한다.
- 주문 상태를 `PAYMENT_FAILED`로 변경한다.
- 예약 재고를 `available_quantity`로 복구한다.
- 결제 실패도 주문 이벤트 로그에 남긴다.

4-3. 시퀀스 다이어그램

```

sequenceDiagram
    actor User
    participant Client
    participant PaymentController
    participant PaymentService
    participant PaymentRepository
    participant OrderRepository
    participant InventoryRepository

```

```

participant OrderEventLogRepository
participant DB

User->>Client: 결제 실패
Client->>PaymentController: POST /api/payments/fail
PaymentController->>PaymentService: failPayment(request)

PaymentService->>PaymentRepository: 결제 조회
PaymentRepository->>DB: SELECT payment WHERE payment_key = ?
DB-->>PaymentRepository: payment READY
PaymentRepository-->>PaymentService: 결제 정보 반환

PaymentService->>PaymentRepository: payment FAILED 변경
PaymentRepository->>DB: UPDATE payment SET status = FAILED, failure_code
= ?

PaymentService->>OrderRepository: order PAYMENT_FAILED 변경
OrderRepository->>DB: UPDATE orders SET status = PAYMENT_FAILED

loop 주문상품별 예약 재고 해제
    PaymentService->>InventoryRepository: releaseReservedStock(productId,
quantity)
    InventoryRepository->>DB: UPDATE inventory SET reserved -= q,
available += q
end

PaymentService->>OrderEventLogRepository: 결제 실패 이벤트 저장
OrderEventLogRepository->>DB: INSERT order_event_log

PaymentService-->>PaymentController: 실패 처리 완료
PaymentController-->>Client: 200 OK
Client-->>User: 결제 실패 안내

```

5. 주문 취소 및 결제 취소

5-1. 목적

사용자가 주문을 취소하면 주문 상태, 결제 상태, 재고 상태를 함께 보장한다.

5-2. 핵심 규칙

- 본인 주문인지 검증한다.
- 배송 시작 전 주문만 일반 취소 가능하다.
- 결제 승인 상태라면 PG 결제 취소를 먼저 수행한다.

- PG 취소 성공 후 주문 상태를 CANCELLED로 변경한다.
- 재고를 복구한다.
- 결제 취소 요청도 Idempotency-Key를 사용한다.

5-3. 시퀀스 다이어그램

```
sequenceDiagram
    actor User
    participant Client
    participant OrderController
    participant OrderService
    participant OrderRepository
    participant DeliveryRepository
    participant PaymentRepository
    participant PG
    participant InventoryRepository
    participant OrderEventLogRepository
    participant DB

    User->>Client: 주문 취소 요청
    Client->>OrderController: POST /api/orders/{orderId}/cancel
    OrderController->>OrderService: cancelOrder(memberId, orderId, reason)

    OrderService->>OrderRepository: 주문 조회
    OrderRepository->>DB: SELECT orders WHERE order_id = ?
    DB-->>OrderRepository: 주문 반환
    OrderRepository-->>OrderService: 주문 반환

    OrderService->>OrderService: 본인 주문 여부 검증
    OrderService->>DeliveryRepository: 배송 상태 조회
    DeliveryRepository->>DB: SELECT delivery WHERE order_id = ?
    DB-->>DeliveryRepository: 배송 상태 READY
    DeliveryRepository-->>OrderService: 배송 정보 반환

    alt 배송 중 또는 배송 완료
        OrderService-->>OrderController: INVALID_ORDER_STATUS
        OrderController-->>Client: 400 취소 불가
        Client-->>User: 취소 불가 안내
    else 취소 가능
        OrderService->>PaymentRepository: 결제 조회
        PaymentRepository->>DB: SELECT payment WHERE order_id = ?
        DB-->>PaymentRepository: payment APPROVED
        PaymentRepository-->>OrderService: 결제 정보 반환

        OrderService->>PG: 결제 취소 요청
        PG-->>OrderService: 결제 취소 성공
```

```

OrderService->>PaymentRepository: payment CANCELLED 변경
PaymentRepository->>DB: UPDATE payment SET status = CANCELLED

OrderService->>OrderRepository: order CANCELLED 변경
OrderRepository->>DB: UPDATE orders SET status = CANCELLED

loop 주문상품별 재고 복구
    OrderService->>InventoryRepository: restoreStock(productId,
quantity)
    InventoryRepository->>DB: UPDATE inventory SET total += q,
available += q
end

OrderService->>OrderEventLogRepository: 취소 이벤트 저장
OrderEventLogRepository->>DB: INSERT order_event_log

OrderService-->>OrderController: 취소 완료
OrderController-->>Client: 200 OK
Client-->>User: 주문 취소 완료
end

```

6. 리뷰 작성

6-1. 목적

리뷰는 실제 구매자만 작성할 수 있으며, 하나의 주문상품에는 하나의 리뷰만 작성할 수 있다.

6-2. 핵심 규칙

- order_item이 본인 주문에 속해야 한다.
- 주문 상태가 DELIVERED 또는 CONFIRMED여야 한다.
- 같은 order_item에는 리뷰를 한 번만 작성할 수 있다.
- 중복 리뷰 방지는 DB UNIQUE 제약조건으로 최종 보장한다.

6-3. 시퀀스 다이어그램

```

sequenceDiagram
    actor User
    participant Client
    participant ReviewController
    participant ReviewService
    participant OrderItemRepository
    participant OrderRepository

```



```
participant ReviewRepository
participant OutboxRepository
participant DB
```

```
User->>Client: 리뷰 작성
```

```
Client->>ReviewController: POST /api/products/{productId}/reviews
```

```
ReviewController->>ReviewService: createReview(memberId, productId,
request)
```

```
ReviewService->>OrderItemRepository: orderItem 조회
```

```
OrderItemRepository->>DB: SELECT order_item WHERE order_item_id = ?
```

```
DB-->>OrderItemRepository: orderItem 반환
```

```
OrderItemRepository-->>ReviewService: orderItem 반환
```

```
ReviewService->>OrderRepository: 주문 조회
```

```
OrderRepository->>DB: SELECT orders WHERE order_id = ?
```

```
DB-->>OrderRepository: 주문 반환
```

```
OrderRepository-->>ReviewService: 주문 반환
```

```
ReviewService->>ReviewService: 본인 주문 여부 확인
```

```
ReviewService->>ReviewService: 주문 상태 DELIVERED 또는 CONFIRMED 확인
```

```
ReviewService->>ReviewRepository: 기존 리뷰 조회
```

```
ReviewRepository->>DB: SELECT review WHERE order_item_id = ?
```

```
DB-->>ReviewRepository: 없음
```

```
ReviewRepository-->>ReviewService: 없음
```

```
ReviewService->>ReviewRepository: 리뷰 저장
```

```
ReviewRepository->>DB: INSERT review
```

```
alt 동시에 중복 INSERT 발생
```

```
DB-->>ReviewRepository: UNIQUE 제약조건 위반
```

```
ReviewRepository-->>ReviewService: DUPLICATE_REVIEW 예외
```

```
ReviewService-->>ReviewController: DUPLICATE_REVIEW
```

```
ReviewController-->>Client: 409 Conflict
```

```
Client-->>User: 이미 리뷰 작성됨 안내
```

```
else 저장 성공
```

```
ReviewService->>OutboxRepository: ReviewCreatedEvent 저장
```

```
OutboxRepository->>DB: INSERT outbox_event
```

```
ReviewService-->>ReviewController: 리뷰 작성 성공
```

```
ReviewController-->>Client: 200 OK
```

```
Client-->>User: 리뷰 작성 완료
```

```
end
```

7. 결제 대기 주문 만료 배치

7-1. 목적

결제 대기 상태가 오래 지속된 주문은 자동 만료 처리하고 예약 재고를 해제한다.

7-2. 핵심 규칙

- PENDING_PAYMENT 상태가 일정 시간 이상 지속된 주문을 대상으로 한다.
- PG에 실제 결제 승인 내역이 있는지 확인한다.
- 결제 승인 내역이 없다면 주문을 PAYMENT_FAILED 또는 CANCELLED로 변경한다.
- 예약 재고를 해제한다.
- 배치 실행 이력과 실패 데이터를 저장한다.

7-3. 시퀀스 다이어그램

```
sequenceDiagram
    participant Scheduler
    participant PendingPaymentExpirationJob
    participant OrderRepository
    participant PaymentRepository
    participant PG
    participant InventoryRepository
    participant BatchRepository
    participant DB

    Scheduler->>PendingPaymentExpirationJob: 배치 실행
    PendingPaymentExpirationJob->>BatchRepository: 배치 실행 이력 STARTED 저장
    BatchRepository->>DB: INSERT batch_job_execution

    PendingPaymentExpirationJob->>OrderRepository: 만료 대상 주문 조회
    OrderRepository->>DB: SELECT orders WHERE status = PENDING_PAYMENT AND created_at < ?
    DB-->>OrderRepository: 주문 목록
    OrderRepository-->>PendingPaymentExpirationJob: 대상 주문 반환

    loop 주문별 처리
        PendingPaymentExpirationJob->>PaymentRepository: 결제 정보 조회
        PaymentRepository->>DB: SELECT payment WHERE order_id = ?
        DB-->>PaymentRepository: payment READY
        PaymentRepository-->>PendingPaymentExpirationJob: 결제 정보 반환

        PendingPaymentExpirationJob->>PG: 실제 결제 승인 여부 조회
        PG-->>PendingPaymentExpirationJob: 승인 내역 없음
```

```
PendingPaymentExpirationJob->>OrderRepository: 주문 PAYMENT_FAILED 변경  
OrderRepository->>DB: UPDATE orders SET status = PAYMENT_FAILED
```

```
PendingPaymentExpirationJob->>PaymentRepository: payment FAILED 변경  
PaymentRepository->>DB: UPDATE payment SET status = FAILED
```

```
loop 주문상품별 예약 재고 해제  
    PendingPaymentExpirationJob->>InventoryRepository:  
releaseReservedStock(productId, quantity)  
    InventoryRepository->>DB: UPDATE inventory SET reserved -= q,  
available += q  
end  
end
```

```
PendingPaymentExpirationJob->>BatchRepository: 배치 실행 이력 COMPLETED 변경  
BatchRepository->>DB: UPDATE batch_job_execution
```

8. 결제 상태 불일치 복구 배치

8-1. 목적

PG 결제는 성공했지만 서버 장애로 인해 주문 상태가 **PAID** 로 변경되지 못한 경우를 복구한다.

8-2. 대표 불일치 상황

```
payment.status = APPROVED  
orders.status = PENDING_PAYMENT
```

또는

```
PG에는 결제 승인 존재  
payment.status = READY  
orders.status = PENDING_PAYMENT
```

8-3. 핵심 규칙

- payment_key 기준으로 PG 결제 상태를 재조회한다.
- PG 승인 내역이 확인되면 payment APPROVED로 보정한다.
- 주문 상태를 PAID로 보정한다.
- 예약 재고를 확정 차감한다.
- 보정 이력을 order_event_log에 남긴다.

8-4. 시퀀스 다이어그램

```
sequenceDiagram
    participant Scheduler
    participant PaymentReconciliationJob
    participant PaymentRepository
    participant OrderRepository
    participant PG
    participant InventoryRepository
    participant OrderEventLogRepository
    participant BatchRepository
    participant DB

    Scheduler->>PaymentReconciliationJob: 배치 실행
    PaymentReconciliationJob->>BatchRepository: 실행 이력 STARTED 저장
    BatchRepository->>DB: INSERT batch_job_execution

    PaymentReconciliationJob->>PaymentRepository: 불일치 후보 조회
    PaymentRepository->>DB: SELECT payment WHERE status IN (READY, REQUESTED)
    DB-->>PaymentRepository: 결제 후보 목록
    PaymentRepository-->>PaymentReconciliationJob: 후보 반환

    loop 결제 후보별 처리
        PaymentReconciliationJob->>PG: paymentKey로 결제 상태 조회
        PG-->>PaymentReconciliationJob: APPROVED

        PaymentReconciliationJob->>PaymentRepository: payment APPROVED 보정
        PaymentRepository->>DB: UPDATE payment SET status = APPROVED

        PaymentReconciliationJob->>OrderRepository: order PAID 보정
        OrderRepository->>DB: UPDATE orders SET status = PAID

        loop 주문상품별 예약 재고 확정
            PaymentReconciliationJob->>InventoryRepository:
confirmReservedStock(productId, quantity)
            InventoryRepository->>DB: UPDATE inventory SET reserved -= q,
total -= q
        end

        PaymentReconciliationJob->>OrderEventLogRepository: 보정 이력 저장
        OrderEventLogRepository->>DB: INSERT order_event_log
    end

    PaymentReconciliationJob->>BatchRepository: 실행 이력 COMPLETED 변경
    BatchRepository->>DB: UPDATE batch_job_execution
```

9. 주문 상태 전이 다이어그램

```
stateDiagram-v2
    [*] --> PENDING_PAYMENT

    PENDING_PAYMENT --> PAID: 결제 승인
    PENDING_PAYMENT --> PAYMENT_FAILED: 결제 실패
    PENDING_PAYMENT --> CANCELLED: 결제 대기 만료

    PAID --> PREPARING: 상품 준비 시작
    PAID --> CANCEL_REQUESTED: 취소 요청

    PREPARING --> SHIPPED: 배송 시작
    PREPARING --> CANCEL_REQUESTED: 취소 요청

    SHIPPED --> DELIVERED: 배송 완료

    DELIVERED --> CONFIRMED: 구매 확정
    DELIVERED --> REFUND_REQUESTED: 환불 요청

    CANCEL_REQUESTED --> CANCELLED: 취소 승인
    REFUND_REQUESTED --> REFUNDED: 환불 완료

    CANCELLED --> [*]
    REFUNDED --> [*]
    CONFIRMED --> [*]
```

10. 결제 상태 전이 다이어그램

```
stateDiagram-v2
    [*] --> READY

    READY --> REQUESTED: 결제 요청
    REQUESTED --> APPROVED: 승인 성공
    REQUESTED --> FAILED: 승인 실패

    APPROVED --> CANCELLED: 전체 취소
    APPROVED --> PARTIAL_CANCELLED: 부분 취소

    PARTIAL_CANCELLED --> CANCELLED: 잔여 금액 전체 취소

    CANCELLED --> REFUNDED: 환불 완료
```

FAILED --> [*]
REFUNDED --> [*]

11. 시퀀스 설계 핵심 요약

1. 주문 생성은 결제 완료가 아니라 PENDING_PAYMENT 주문과 예약 재고를 만든다.
2. 결제 승인 성공 시 주문을 PAID로 바꾸고 예약 재고를 확정한다.
3. PG 콜백은 중복으로 들어올 수 있으므로 멱등하게 처리한다.
4. 결제 실패 시 예약 재고를 반드시 해제한다.
5. 주문 취소는 주문 상태, 배송 상태, 결제 상태를 함께 확인한다.
6. 리뷰 작성은 실제 구매자만 가능하며 DB UNIQUE로 중복을 막는다.
7. 결제 대기 만료 배치는 예약 재고 누수를 방지한다.
8. 결제 상태 복구 배치는 PG와 내부 DB 상태 불일치를 보정한다.

운영 가능한 쇼핑몰의 시퀀스 다이어그램은 정상 흐름뿐 아니라 **중복 요청, 실패, 만료, 복구** 흐름까지 포함해야 한다.